



Nexus Vault Security Audit Report

Prepared by: Saurabh Kaplas

Date: 2025-12-25
Engagement: Time-boxed security review (48 hours): smart contracts & oracle interactions only
Files Reviewed: `NexusVault.sol`, `ElasticSupplyToken.sol` (EST), `TWAPOracle.sol`, and supporting interfaces/helpers

1. Purpose & Scope

A focused security review of the Nexus Vault system was conducted to evaluate:

- The ERC-4626 vault implementation (`NexusVault.sol`)
- Interactions with the elastic supply rebase token (`ElasticSupplyToken.sol`)
- TWAP oracle integration (`TWAPOracle.sol`) and its security model

Key focus areas:

- Economic attacks (ERC-4626 inflation, rebase front-running, oracle timing manipulation)
- Cross-protocol risks (share-price manipulation via composable vault shares)
- Logic flaws in accounting for fee-on-transfer token behavior

The engagement was limited to the artifacts listed above.

2. Executive Summary

Critical Findings

ID	Title	Severity
1	Missing approval check in <code>withdraw</code> / <code>redeem</code>	Critical
2	ERC-4626 first-depositor / donation inflation	High
3	Misaccounting for EST's dynamic fee-on-transfer	High
4	Rebase front-running vulnerability	High
5	Share-price manipulation via stealth donations (cross-protocol)	Medium
6	Permissionless TWAP <code>forceUpdate()</code> (oracle timing manipulation)	Medium
7	Withdrawal griefing / sandwich risk	Medium
8	Precision loss / zero-shares edge case	Medium
9	Governance & ownership hygiene (two-step ownership transfer)	Medium
10	Missing event emissions for critical state changes	Medium

ID	Title	Severity
11	Unnecessary SafeMath import in Solidity 0.8+	Low
12	Public functions should be external (gas optimization)	Low
13	Inconsistent error handling / missing custom errors	Low

Key Takeaways

- **Immediate action required:** The `withdraw` and `redeem` functions lack approval checks, creating a direct theft vector for any token holder's funds.
- **Accounting defects:** The vault does not account for EST's dynamic fee-on-transfer behavior, risking under-collateralization.
- **Oracle control:** Permissionless oracle updates enable attackers to manipulate the conditions triggering rebase and fee mechanics.
- **Design risks:** ERC-4626 composability without sanitized accounting creates cascading risks in downstream protocols.

Disclaimer

The "SK" team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

We use the [Halborn](#) severity matrix to determine severity. See the documentation for more details.

3. Methodology

This audit employed the following approach:

- **White-box manual code review:** Line-by-line analysis of all provided Solidity sources
- **Static analysis tools:** Automated scanning for common vulnerabilities, gas inefficiencies, and code quality issues
- **Specification compliance:** Validation against ERC-4626 and ERC-20 standards
- **Economic threat modeling:** Analysis of rebase, fee, and oracle interaction vectors
- **Proof-of-concept construction:** Concise, reproducible exploits in ethers.js and Solidity
- **Cross-system analysis:** Integration of related findings across smart contracts and oracle mechanics

4. Detailed Findings

Finding 1: Missing Approval Check in `withdraw` / `redeem` : **Critical**

Description

The `withdraw(assets, receiver, owner)` and `redeem(shares, receiver, owner)` functions accept an `owner` parameter but do **not enforce ERC-4626 allowance semantics** when `msg.sender != owner`. The vault burns the owner's shares without verifying the caller's authorization, enabling arbitrary share burning and fund theft.

Root Cause

Missing approval/allowance check before burning shares from an account other than the caller.

Proof of Concept

```
// Attacker drains victim's vault shares
const victimShares = await vault.balanceOf(victimAddress);
await vault.withdraw(
  ethers.utils.parseUnits("1.0", 18), // assets
  attackerAddress,                     // receiver
  victimAddress                        // owner (unauthorized)
);
// Attacker receives assets; victim's shares burned without consent
```

Impact

Severity: Critical

Scope: Any token holder in the vault

Risk: Immediate theft of funds: attackers can withdraw and redirect assets from any account without authorization.

Recommendation

Enforce ERC-4626 allowance semantics when `msg.sender != owner`:

```
function withdraw(
  uint256 assets,
  address receiver,
  address owner
) public virtual override returns (uint256 shares) {
  shares = previewWithdraw(assets);

  if (msg.sender != owner) {
    uint256 allowed = allowance(owner, msg.sender);
    require(allowed >= shares, "ERC4626: insufficient approval");
    _approve(owner, msg.sender, allowed - shares);
  }
  _withdraw(msg.sender, receiver, owner, assets, shares);
  return shares;
}

function redeem(
```

```
    uint256 shares,
    address receiver,
    address owner
) public virtual override returns (uint256 assets) {
    assets = previewRedeem(shares);

    if (msg.sender != owner) {
        uint256 allowed = allowance(owner, msg.sender);
        require(allowed >= shares, "ERC4626: insufficient approval");
        _approve(owner, msg.sender, allowed - shares);
    }
    _withdraw(msg.sender, receiver, owner, assets, shares);
    return assets;
}
```

Finding 2: ERC-4626 First-Depositor / Donation Inflation : **High**

Description

If an attacker transfers EST tokens directly to the vault address (bypassing `deposit()`), the `totalAssets()` increases while `totalSupply()` remains unchanged. Subsequent honest deposits mint very few or zero shares, allowing the attacker to capture value disproportionately.

Root Cause

The vault's `totalAssets()` uses `balanceOf(address(this))` without distinguishing between assets received via `deposit()` and unsolicited transfers.

Proof of Concept

```
// 1. Vault is empty
// 2. Attacker transfers EST directly to vault
await est.transfer(vault.address, ethers.utils.parseUnits("20000", 18));

// 3. Honest depositor calls deposit()
await est.approve(vault.address, ethers.utils.parseUnits("10000", 18));
const tx = await vault.deposit(
    ethers.utils.parseUnits("10000", 18),
    honestUserAddress
);

// 4. Honest depositor receives near-zero shares due to inflated share price
const receipt = await tx.wait();
// Expected: ~10000 shares (1:1 with no prior deposits)
// Actual: ~500 shares (diluted by attacker's donation)
```

Impact

Severity: High

Scope: All vault depositors

Risk: Depositors receive fewer shares than expected, loss of capital, and concentration of value in attacker's hands.

Recommendation

Option A: Seed Vault at Deployment (Quick Fix)

```
constructor(  
    IERC20 _asset,  
    string memory _name,  
    string memory _symbol  
) ERC4626(_asset) ERC20(_name, _symbol) {  
    // Mint minimal shares to prevent 1:1 first-deposit attacks  
    _mint(address(this), 1);  
}
```

Option B: Track Internal Deposits (**lockedAssets**)

Maintain an internal accounting of only the assets received through authorized **deposit()** calls:

```
uint256 private lockedAssets;  
  
function deposit(uint256 assets, address receiver)  
    public  
    override  
    returns (uint256 shares)  
{  
    uint256 before = asset.balanceOf(address(this));  
    asset.transferFrom(msg.sender, address(this), assets);  
    uint256 after = asset.balanceOf(address(this));  
    uint256 actualReceived = after - before;  
  
    require(actualReceived > 0, "No assets received");  
  
    lockedAssets += actualReceived;  
  
    if (totalSupply() == 0) {  
        shares = actualReceived; // 1:1 for first deposit  
    } else {  
        shares = (actualReceived * totalSupply()) / lockedAssets;  
    }  
  
    _mint(receiver, shares);  
    emit Deposit(msg.sender, receiver, assets, shares);  
    return shares;  
}  
  
function totalAssets() public view override returns (uint256) {
```

```
        return lockedAssets;
    }

    function sweepDonations(address treasury) external onlyOwner {
        uint256 bal = asset.balanceOf(address(this));
        if (bal > lockedAssets) {
            uint256 donation = bal - lockedAssets;
            asset.transfer(treasury, donation);
        }
    }
}
```

Recommendation: Implement Option B for long-term robustness. Option A is acceptable as a temporary measure.

Finding 3: Misaccounting for EST's Dynamic Fee-on-Transfer: **High**

Description

EST implements a dynamic transfer fee when price remains below peg for more than 48 hours. The vault's `deposit()` and `withdraw()` functions assume the full `assets` parameter is transferred, but if a fee applies, fewer tokens are actually received. This causes the vault to:

1. Mint shares for the requested (pre-fee) amount
2. Receive fewer tokens than assumed
3. Become under-collateralized

Root Cause

No verification that the actual amount received matches the expected amount after `transferFrom()`.

Proof of Concept

```
uint256 assets = 100e18;
uint256 before = asset.balanceOf(address(this));

// EST applies a 1% fee in this scenario
asset.transferFrom(msg.sender, address(this), assets);

uint256 after = asset.balanceOf(address(this));
uint256 actualReceived = after - before; // 99e18 (1% fee)

// But vault minted shares based on 'assets' (100e18)
// Vault is now 1% under-collateralized
```

Impact

Severity: High

Scope: All vault participants

Risk: Vault becomes insolvent; redemptions may fail or users receive fewer assets than expected.

Recommendation

Primary Fix: Measure and use the actual amount received:

```
function deposit(uint256 assets, address receiver)
    public
    override
    returns (uint256 shares)
{
    require(assets > 0, "Must deposit > 0");

    // Record balance before
    uint256 before = asset.balanceOf(address(this));

    // Transfer from caller
    asset.transferFrom(msg.sender, address(this), assets);

    // Record balance after
    uint256 after = asset.balanceOf(address(this));
    uint256 actualReceived = after - before;

    // Ensure something was received
    require(actualReceived > 0, "No assets received (fee applied)");

    // Use actual received amount for share calculation
    shares = _convertToShares(actualReceived, totalSupply());

    require(shares > 0, "Shares minted must be > 0");

    _mint(receiver, shares);
    emit Deposit(msg.sender, receiver, actualReceived, shares);

    return shares;
}

function withdraw(
    uint256 assets,
    address receiver,
    address owner
) public override returns (uint256 shares) {
    shares = previewWithdraw(assets);

    // ... allowance checks ...

    _withdraw(msg.sender, receiver, owner, assets, shares);

    uint256 before = asset.balanceOf(address(this));
    asset.transfer(receiver, assets);
```



```
uint256 after = asset.balanceOf(address(this));

require(before - after >= assets * 99 / 100, "Transfer fee too high");

return shares;
}
```

Secondary Mitigation:

If fee-on-transfer tokens are unpredictable or frequent, consider:

- Wrapping EST in a standard ERC-20 that normalizes transfer semantics
- Explicitly prohibiting fee-on-transfer tokens
- Implementing a configurable fee tolerance with explicit user slippage parameters

Finding 4: Rebase Front-Running Vulnerability: **High**

Description

The `rebase()` function is callable without sufficient time-based or randomness-based protection. Attackers can predict or manipulate rebase timing to:

- Deposit before a positive rebase to capture yield unfairly
- Withdraw before a negative rebase to avoid losses
- Sandwich legitimate users' transactions around rebase events

Root Cause

The rebase mechanism lacks temporal gating or randomness protection, making rebase timing predictable and exploitable.

Proof of Concept

```
// Attacker monitors pending rebase() calls
// Deposits immediately before positive rebase
await est.approve(vault.address, ethers.utils.parseUnits("1000000", 18));
await vault.deposit(ethers.utils.parseUnits("1000000", 18), attackerAddress);

// Rebase increases all balances by 10%
await vault.rebase();

// Attacker immediately withdraws, capturing entire 10% gain
await vault.withdraw(
  ethers.utils.parseUnits("1100000", 18),
  attackerAddress,
  attackerAddress
);
```

Impact

Severity: High

Scope: All vault participants

Risk: Legitimate users experience dilution; attackers extract unearned yield; economic incentives become misaligned.

Recommendation

Restrict `rebase()` to an authorized keeper and enforce minimum interval checks:

```
address public rebaseKeeper;
uint256 public nextRebaseTime;
uint256 public constant REBASE_INTERVAL = 1 days;

event RebaseKeeperUpdated(address indexed newKeeper);
event Rebase(uint256 indexed timestamp, uint256 newSupply);

modifier onlyRebaseKeeper() {
    require(msg.sender == rebaseKeeper, "rebase: not keeper");
    _;
}

function setRebaseKeeper(address _keeper) external onlyOwner {
    require(_keeper != address(0), "rebase: invalid keeper");
    rebaseKeeper = _keeper;
    emit RebaseKeeperUpdated(_keeper);
}

function rebase() external onlyRebaseKeeper {
    require(block.timestamp >= nextRebaseTime, "rebase: too early");

    nextRebaseTime = block.timestamp + REBASE_INTERVAL;

    int256 supply = int256(totalSupply());
    int256 targetSupply = int256(_computeTargetSupply());
    int256 delta = targetSupply - supply;

    if (delta > 0) {
        _mint(address(this), uint256(delta));
    } else if (delta < 0) {
        _burn(address(this), uint256(-delta));
    }

    emit Rebase(block.timestamp, uint256(targetSupply));
}
```

Additional mitigations:

- Implement per-block deposit/withdrawal limits around rebase events

- Add commit-reveal mechanism for rebase computations
- Enforce minimum notice period before rebase execution

Finding 5: Share-Price Manipulation via Stealth Donations (Cross-Protocol): **Medium**

Description

Direct transfers of EST to the vault increase `balanceOf(address(this))` and therefore the calculated share price. If vault shares are used as collateral or pricing references in other protocols, attackers can:

1. Donate a large amount of EST to the vault
2. Inflate the vault's apparent value and share price
3. Trigger liquidations or bad debt in downstream lending/DEX protocols

Root Cause

Lack of distinction between assets received via `deposit()` and unsolicited transfers; downstream protocols rely on naïve `totalAssets()` without sanitization.

Proof of Concept

```
// Attacker donates EST to vault
await est.transfer(vault.address, ethers.utils.parseUnits("1000000", 18));

// Vault's share price artificially inflates
const sharePrice = await vault.convertToAssets(ethers.utils.parseUnits("1", 18));
// sharePrice now includes the donation

// External protocol (lending, AMM, etc.) uses vault's share price for collateral
// valuation
// Attacker's collateral is over-valued → triggered liquidation of legitimate
// users
```

Impact

Severity: Medium

Scope: External protocols using vault shares for pricing

Risk: Cascading insolvencies, bad debt, and liquidation of users in dependent protocols.

Recommendation

Implement the `lockedAssets` tracking and `sweepDonations()` policy described in **Finding 2**:

```
function totalAssets() public view override returns (uint256) {
    // Use only locked (deposited) assets, not all held assets
    return lockedAssets;
}
```

```
function sweepDonations(address treasury) external onlyOwner {
    uint256 held = asset.balanceOf(address(this));
    uint256 expected = lockedAssets + _computeAccruedYield();

    if (held > expected) {
        uint256 donation = held - expected;
        asset.transfer(treasury, donation);
        emit DonationSwept(treasury, donation);
    }
}
```

External Safeguards:

- Publish a sanitized oracle for vault share prices that excludes unsolicited transfers
- Implement rate limits on share price changes to detect manipulation
- Require downstream protocols to use explicit price feeds (Chainlink/aggregated) rather than vault's internal `totalAssets()`

Finding 6: Permissionless TWAP `forceUpdate()`: Medium

Description

The TWAP oracle exposes a public `forceUpdate()` function callable by any user. This allows attackers to:

- Reset the TWAP observation window at strategic times
- Prevent or ensure the "48-hour below-peg" condition that triggers EST's dynamic fee
- Manipulate the price signal used to decide rebase behavior

Root Cause

Missing access control on `forceUpdate()`.

Proof of Concept

```
// Attacker prevents fee activation by forcing a TWAP reset
await twapOracle.forceUpdate();

// Later, manipulates timing to trigger undesirable rebase
// by forcing another update at a critical moment
await twapOracle.forceUpdate();
```

Impact

Severity: Medium

Scope: Token economics and vault participants

Risk: Attackers control the conditions triggering rebase and fee mechanics, undermining the token's intended economic behavior and creating arbitrage opportunities.

Recommendation

Restrict `forceUpdate()` to an authorized keeper address:

```
address public keeper;
uint256 public minForceUpdateInterval = 10 minutes;
uint256 private lastForceUpdate;

event KeeperUpdated(address indexed newKeeper);
event OracleUpdated(uint256 indexed timestamp, uint256 newPrice);

modifier onlyKeeper() {
    require(msg.sender == keeper, "TWAPOracle: caller is not keeper");
    _;
}

function setKeeper(address newKeeper) external onlyOwner {
    require(newKeeper != address(0), "Invalid keeper");
    keeper = newKeeper;
    emit KeeperUpdated(newKeeper);
}

function forceUpdate() external onlyKeeper {
    require(
        block.timestamp - lastForceUpdate >= minForceUpdateInterval,
        "TWAPOracle: update too soon"
    );

    lastForceUpdate = block.timestamp;
    uint256 newPrice = _computeTWAP();
    _recordObservation(newPrice);

    emit OracleUpdated(block.timestamp, newPrice);
}
```

Best Practice: Assign the keeper role to a multisig or governance contract with timelock delays for critical operations.

Finding 7: Withdrawal Griefing / Sandwich Risk: **Medium**

Description

An attacker can exploit the combination of permissionless oracle updates and vault withdrawals to sandwich victims:

1. Monitor pending withdrawal transactions

2. Call `forceUpdate()` to manipulate oracle price
3. User's withdrawal experiences excessive slippage
4. Attacker profits from the price movement

Root Cause

Lack of slippage protection in `withdraw()/redeem()` and permissionless oracle control.

Proof of Concept

```
// Victim initiates withdrawal with 1% slippage tolerance
const tx = await victim.vault.withdraw(
  ethers.utils.parseUnits("1000", 18),
  victimAddress,
  victimAddress
);

// Attacker front-runs with oracle manipulation
await twapOracle.forceUpdate(); // triggers fee or price change

// Victim's withdrawal slips beyond tolerance and fails
// Or succeeds but victim receives less than expected
```

Impact

Severity: Medium-High

Scope: Withdrawal transactions

Risk: Users unable to withdraw; capital becomes locked or users suffer unintended losses.

Recommendation

Implement slippage protection and restrict oracle control:

```
function withdraw(
  uint256 assets,
  address receiver,
  address owner,
  uint256 minAssets // slippage protection
) public returns (uint256 shares) {
  require(assets >= minAssets, "Insufficient output");

  uint256 shares = previewWithdraw(assets);

  if (msg.sender != owner) {
    uint256 allowed = allowance(owner, msg.sender);
    require(allowed >= shares, "ERC4626: insufficient allowance");
    _approve(owner, msg.sender, allowed - shares);
  }
}
```

```
    _burn(owner, shares);
    asset.transfer(receiver, assets);

    emit Withdraw(msg.sender, receiver, owner, assets, shares);
    return shares;
}

function redeem(
    uint256 shares,
    address receiver,
    address owner,
    uint256 minAssets // slippage protection
) public returns (uint256 assets) {
    assets = previewRedeem(shares);
    require(assets >= minAssets, "Insufficient output");

    if (msg.sender != owner) {
        uint256 allowed = allowance(owner, msg.sender);
        require(allowed >= shares, "ERC4626: insufficient allowance");
        _approve(owner, msg.sender, allowed - shares);
    }

    _burn(owner, shares);
    asset.transfer(receiver, assets);

    emit Withdraw(msg.sender, receiver, owner, assets, shares);
    return assets;
}
```

Finding 8: Precision Loss / Zero-Shares Edge Case: **Medium**

Description

Small deposits may convert to zero shares due to integer division truncation in the `_convertToShares()` function. Users can deposit assets and receive no shares in return, effectively losing their deposit.

Root Cause

Insufficient validation that share calculation does not result in zero. For example:

```
shares = (assets * totalSupply()) / totalAssets();
// If (assets * totalSupply()) < totalAssets(), shares == 0
```

Proof of Concept

```
// Assuming vault has large TVL
const largeVault = ethers.utils.parseUnits("1000000", 18);
await vault.deposit(largeVault);

// Attacker deposits tiny amount
const tinyDeposit = ethers.utils.parseUnits("1", 6); // 1 token with 6 decimals
const shares = await vault.deposit(tinyDeposit, attackerAddress);

// shares == 0, attacker lost the deposit
console.log(shares); // 0
```

Impact

Severity: Medium

Scope: Users attempting small deposits

Risk: Loss of user funds; poor user experience; potential accumulation of dust.

Recommendation

Enforce minimum deposit amounts and require non-zero share outcome:

```
uint256 public constant MIN_DEPOSIT = 1e6; // Minimum 1 token in base units

function deposit(uint256 assets, address receiver)
    public
    override
    returns (uint256 shares)
{
    require(assets >= MIN_DEPOSIT, "deposit: amount too small");

    uint256 before = asset.balanceOf(address(this));
    asset.transferFrom(msg.sender, address(this), assets);
    uint256 after = asset.balanceOf(address(this));
    uint256 actualReceived = after - before;

    require(actualReceived > 0, "No assets received");

    shares = _convertToShares(actualReceived);

    // Ensure shares minted is non-zero
    require(shares > 0, "deposit: shares must be > 0");

    _mint(receiver, shares);
    emit Deposit(msg.sender, receiver, actualReceived, shares);

    return shares;
}

function redeem(uint256 shares, address receiver, address owner)
    public
```



```
    override
    returns (uint256 assets)
    {
        require(shares > 0, "redeem: shares must be > 0");

        assets = _convertToAssets(shares);
        require(assets > 0, "redeem: assets must be > 0");

        // ... rest of redemption logic ...
    }
```

Finding 9: Governance & Ownership Hygiene: **Medium**

Description

The contract uses single-step `transferOwnership()`, which risks accidental transfer to an invalid address or loss of control if the new owner address is misconfigured. Once transferred, there is no recovery mechanism.

Root Cause

Missing confirmation mechanism for ownership transfer.

Proof of Concept

```
// Owner mistypes new owner address
await vault.transferOwnership("0x0000000000000000000000000000000000000000000000000000000000000001"); //
OOPS

// Ownership transferred to invalid address
// No recovery possible
```

Impact

Severity: Medium

Scope: Contract governance and control

Risk: Loss of administrative control; emergency pause/recovery functions become inaccessible; inability to update critical parameters.

Recommendation

Implement two-step ownership transfer pattern:

```
address private _owner;
address private _pendingOwner;

event OwnershipTransferred(address indexed previousOwner, address indexed
```

```
newOwner);
event OwnershipTransferInitiated(address indexed currentOwner, address indexed
pendingOwner);

modifier onlyOwner() {
    require(msg.sender == _owner, "Ownable: caller is not owner");
    _;
}

function owner() public view returns (address) {
    return _owner;
}

function pendingOwner() public view returns (address) {
    return _pendingOwner;
}

function transferOwnership(address newOwner) external onlyOwner {
    require(newOwner != address(0), "Ownable: zero address");
    _pendingOwner = newOwner;
    emit OwnershipTransferInitiated(_owner, newOwner);
}

function acceptOwnership() external {
    require(msg.sender == _pendingOwner, "Ownable: not pending owner");
    address oldOwner = _owner;
    _owner = msg.sender;
    _pendingOwner = address(0);
    emit OwnershipTransferred(oldOwner, msg.sender);
}

function renounceOwnership() external onlyOwner {
    _pendingOwner = address(0);
    emit OwnershipTransferred(_owner, address(0));
    _owner = address(0);
}
```

Best practices:

- Always emit events for ownership changes
- Require explicit acceptance from new owner
- Document the transfer process in user guides
- Implement multisig or timelock for critical operations

Finding 10: Missing Event Emissions for Critical State Changes: **Medium**

Description

The `rebase()` function modifies the critical `supplyFactor` state variable which affects all token balances across the system, but does not emit an event. Similarly, oracle `forceUpdate()` executes without event

logging. This makes it impossible to track these critical operations off-chain, breaking composability and hindering debugging.

Root Cause

Missing event declarations and emissions for state-altering operations.

Impact

Severity: Medium

Scope: Off-chain systems, auditing, and debugging

Risk: Off-chain indexers cannot detect rebases; external protocols cannot react to state changes; forensic analysis becomes difficult; governance transparency is reduced.

Recommendation

Emit events for all critical state changes:

```
event Rebase(uint256 indexed timestamp, uint256 oldSupplyFactor, uint256
newSupplyFactor, uint256 price);
event OracleUpdated(uint256 indexed timestamp, uint256 price);
event FeesActivated(uint256 timestamp);
event FeesDeactivated(uint256 timestamp);

function rebase() external onlyRebaseKeeper {
    require(block.timestamp >= nextRebaseTime, "rebase: too early");

    nextRebaseTime = block.timestamp + REBASE_INTERVAL;

    uint256 oldFactor = supplyFactor;
    uint256 currentPrice = oracle.getLatestPrice();

    // ... rebase logic ...

    emit Rebase(block.timestamp, oldFactor, supplyFactor, currentPrice);
}

function forceUpdate() external onlyKeeper {
    require(
        block.timestamp - lastForceUpdate >= minForceUpdateInterval,
        "TWAPOracle: update too soon"
    );

    lastForceUpdate = block.timestamp;
    uint256 newPrice = _computeTWAP();
    _recordObservation(newPrice);

    emit OracleUpdated(block.timestamp, newPrice);
}
```

Finding 11: Unnecessary SafeMath Import in Solidity 0.8+ : **Low**

Description

The **TWAPOracle** contract imports and uses the **SafeMath** library despite using Solidity **^0.8.0**, which includes built-in overflow/underflow protection. This adds unnecessary gas costs and code complexity.

Impact

Severity: Low

Risk: Increased gas costs for operations using **SafeMath** (approximately 100–200 extra gas per operation) without security benefit.

Recommendation

Remove SafeMath and use native arithmetic:

```
// REMOVE: import "@openzeppelin/contracts/utils/math/SafeMath.sol";
// REMOVE: using SafeMath for uint256;

// BEFORE (with SafeMath)
return currentPrice < PEG_PRICE && (block.timestamp.sub(lastUpdateTimestamp) >
BELOW_PEG_DURATION);

// AFTER (native Solidity 0.8+)
return currentPrice < PEG_PRICE && (block.timestamp - lastUpdateTimestamp >
BELOW_PEG_DURATION);
```

Finding 12: Public Functions Should Be External (Gas Optimization): **Low**

Description

Several view functions in **NexusVault** are marked as **public** but are never called internally. Marking them as **external** saves gas by avoiding memory allocation, as external functions read arguments directly from calldata.

Examples

```
// Current (public) - more expensive
function maxDeposit(address) public view override returns (uint256) {
    return type(uint256).max;
}

function maxMint(address) public view override returns (uint256) {
    return type(uint256).max;
}
```

```
function maxWithdraw(address owner) public view override returns (uint256) {
    return balanceOf(owner);
}

function maxRedeem(address owner) public view override returns (uint256) {
    return balanceOf(owner);
}
```

Recommendation

Change to **external**:

```
function maxDeposit(address) external view override returns (uint256) {
    return type(uint256).max;
}

function maxMint(address) external view override returns (uint256) {
    return type(uint256).max;
}

function maxWithdraw(address owner) external view override returns (uint256) {
    return balanceOf(owner);
}

function maxRedeem(address owner) external view override returns (uint256) {
    return balanceOf(owner);
}
```

Impact

Gas savings of approximately 100–200 wei per external call.

Finding 13: Inconsistent Error Handling / Missing Custom Errors: **Low**

Description

The codebase uses inconsistent error messages and patterns. Some operations use generic **require()** strings while others use specific messages. Custom errors (introduced in Solidity 0.8.4) are not used, resulting in higher gas costs for error strings.

Current Issues

```
// Inconsistent patterns:
require(asset.transferFrom(msg.sender, address(this), assets), "Asset transfer failed");
require(shares <= balanceOf(owner), "ERC4626: redeem shares exceeds balance");
require(msg.sender == owner, "Ownable: caller is not the owner");
```

```
// Generic messages don't help debugging
// String errors cost more gas than custom errors
```

Recommendation

Define and use custom errors consistently:

```
// Define custom errors at contract level
error TransferFailed();
error InsufficientShares(uint256 requested, uint256 available);
error Unauthorized();
error InvalidAmount();
error BelowMinimumDeposit();

// Use throughout contract
if (!asset.transferFrom(msg.sender, address(this), assets)) {
    revert TransferFailed();
}

if (shares > balanceOf(owner)) {
    revert InsufficientShares(shares, balanceOf(owner));
}

if (msg.sender != owner) {
    revert Unauthorized();
}

if (assets == 0) {
    revert InvalidAmount();
}

if (assets < MIN_DEPOSIT) {
    revert BelowMinimumDeposit();
}
```

Benefits

- Gas savings of approximately 50–100 wei per failed operation
- Clearer error semantics for debugging
- Consistent error handling patterns across codebase

5. Threat Model & Design Review

System Overview

Nexus Vault combines an ERC-4626 vault wrapper with an elastic supply token (EST) whose supply and transfer fees are governed by a TWAP oracle. The system's security depends on:

- Correct share/asset accounting
- Robust and manipulation-resistant oracle design
- Controlled and auditable governance over rebase/fee parameters

Top Three Non-Code Risks

1. Oracle Manipulation & Centralization

EST's rebase and fee logic depend on a single DEX TWAP that can be force-updated by any user. If the TWAP or underlying DEX liquidity is manipulable, the token's behavior becomes controllable by attackers.

Mitigations:

- Use multi-source oracles (Chainlink, Uniswap V3 TWAP with time-weighted snapshots)
- Restrict `forceUpdate()` to authorized keepers
- Implement circuit breakers and sanity checks on price movements
- Multisig control over critical parameters

2. Economic Fragility from Rebase + Fee Mechanics

Rebase + dynamic fee mechanisms can create feedback loops:

- High fees reduce liquidity
- Low liquidity → price drops further
- Dropping price → more rebases/fees
- Spiral toward insolvency in low-liquidity scenarios

Mitigations:

- Cap rebase magnitude and frequency
- Implement damping mechanisms for gradual adjustments
- Add circuit breakers (halt rebases if price falls >X%)
- Require governance approval for parameter changes above thresholds

3. Composability Risks

ERC-4626 shares are inherently composable. Using unsanitized `totalAssets()` for external pricing allows donation/rebase manipulations to propagate, causing liquidations and bad debt in dependent protocols.

Mitigations:

- Maintain internal accounting (`lockedAssets`) to exclude unsolicited transfers
- Provide sanitized price oracles for share valuation
- Require external protocols to use explicit price feeds
- Publish clear documentation on safe composability patterns

6. Remediation Plan

Immediate

- **Patch `withdraw/redeem`:** Add approval check before burning owner's shares (Critical, Finding 1)
- **Fix deposit accounting:** Measure `balanceBefore/balanceAfter`; use `actualReceived` for share minting (High, Finding 3)
- **Handle fee-on-transfer:** Either wrap EST or explicitly reject fee-on-transfer tokens (High, Finding 3)

Near Term

- **Prevent donation inflation:** Implement `lockedAssets` tracking or seed vault with minimal shares (High, Finding 2)
- **Restrict oracle access:** Lock `forceUpdate()` to authorized keepers; add min interval (Medium, Finding 6)
- **Restrict rebase calls:** Lock `rebase()` to authorized keeper; enforce interval (High, Finding 6)
- **Add slippage protection:** Implement `minAssets/maxAssets` parameters in `withdraw/redeem` (Medium, Finding 7)
- **Implement `sweepDonations()`:** Route unexpected transfers to treasury (Medium, Finding 5)

Governance & Operations

- **Add Pausable & multisig:** Implement emergency pause and multisig approval for critical functions
- **Timelock for parameters:** Require timelock delays for changes to rebase, fee, or keeper assignments
- **Two-step ownership:** Implement `transferOwnership()` + `acceptOwnership()` pattern (Medium, Finding 9)
- **Event emissions:** Add `Rebase`, `OracleUpdate`, `DonationSwept`, `KeeperChanged` events for observability

Testing & Monitoring

- **Unit tests:** Reproduce each PoC; verify fixes work correctly
- **Integration tests:** Test rebase + fee mechanics under various price scenarios
- **Stress tests:** Low liquidity scenarios; large deposit/withdrawal sequences; rapid oracle updates

Runtime monitoring: Alert on large unsolicited transfers; track delta between `balanceOf` and `lockedAssets`

7. Limitations & Assumptions

- **Timebox:** This audit was limited to 48 hours. Deeper formal verification, extensive fuzzing, and detailed economic simulation would further increase confidence.
 - **Scope:** Analysis covers provided source files only; no deployed bytecode verification on-chain.
 - **Oracle assumptions:** Impact scenarios assume moderate DEX liquidity susceptible to manipulation by a well-capitalized attacker.
 - **EST behavior:** Analysis assumes the dynamic fee-on-transfer mechanism is active during certain conditions (below-peg for >48 hours).
-

8. Key Metrics Summary

Category	Count	Details
----------	-------	---------

Category	Count	Details
Critical	1	Missing approval checks in withdraw/redeem
High	3	Donation inflation, fee-on-transfer misaccounting, rebase front-running
Medium	6	Oracle manipulation, stealth donations, precision loss, ownership hygiene, withdrawal griefing, missing events
Low	3	SafeMath import, public vs external, error handling
Total	13	Comprehensive coverage of smart contract, economic, and optimization risks